Contrato técnico

Reto 02

Cifrado de información, tratamiento de ficheros e integración de clientes

DATOS DEL DOCUMENTO	VALOR
Equipo	Enigma
Versión	1.1 (revisión tras observaciones)
Fecha	28 de octubre de 2026
Estado	Propuesta corregida pendiente de validación y aprobación
Ámbito	Python CLI · JavaFX · Android · Unity 6.3
Integrantes	Carlos · Koldo · Kenneth

1. Propósito y alcance

El proyecto propone una solución conjunta compuesta por cuatro aplicaciones autónomas desarrolladas en Python 3.14, JavaFX sobre Java 25 LTS, Android y Unity 6.3 para cifrado, descifrado, tratamiento de ficheros y un minijuego educativo de César. El equipo Enigma está formado por Carlos, Koldo y Kenneth.

Cada aplicación incorporará y ejecutará localmente su propia lógica funcional y criptográfica. Python, JavaFX y Android compartirán una especificación común de contenedor R2AE, lo que permitirá intercambiar y recuperar ficheros entre componentes compatibles sin depender de servicios externos.

Componente	Responsabilidad principal
Python (CLI)	Aplicación por consola con cifrado y descifrado Vigenère y AES.
JavaFX	Cliente gráfico de escritorio para cifrar y descifrar ficheros mediante AES.
Android (Kotlin)	Aplicación Auror de consulta y descifrado de imágenes AES y mensajes César.
Unity 6.3	Minijuego 2D de descifrado César con enemigo, puerta y estados de partida.

La fase Discovery incluye decisiones de arquitectura, planificación contrastable, análisis previo de viabilidad y coordinación con los módulos implicados. Las mediciones de rendimiento, memoria y recuperación de ficheros se efectuarán durante el desarrollo de las aplicaciones, una vez aprobado el contrato; este documento no presenta resultados todavía no obtenidos.

1.1. Arquitectura local e integración funcional

> Se adopta una arquitectura formada por cuatro aplicaciones autónomas. Cada una contendrá el código necesario para ejecutar sus funciones sin comunicarse con un servicio central. Python implementará Vigenère y AES en su CLI; implementará AES en el escritorio; Android implementará descifrado AES y César; Unity implementará sus desafíos César, estados y mecánica 2D dentro del propio juego.

La integración técnica entre Python, JavaFX y Android se realizará por interoperabilidad de archivos AES con formato R2AE, parámetros criptográficos compartidos y pruebas cruzadas de recuperación de bytes. La transferencia se efectuará por almacenamiento local, USB, Bluetooth o las funciones nativas de compartir. Unity formará parte de la misma entrega y aplicará el descifrado César en un minijuego autónomo.

ARQUITECTURA DEFINIDA POR EL EQUIPO: toda la lógica funcional y de cifrado o descifrado se ejecuta dentro de cada aplicación. La integración se documentará y demostrará con el intercambio de ficheros R2AE y la coherencia funcional del conjunto. El contrato completo seguirá sujeto a revisión y aprobación técnica conforme al pliego.

2. Solución e integración

La solución distribuye las responsabilidades entre cuatro aplicaciones autónomas. Cada una ejecutará su código y procesará los datos localmente, de acuerdo con la arquitectura descrita en el apartado 1.1.

2.1. Python CLI

La aplicación dispondrá de un menú interactivo para cifrar y descifrar mediante Vigenère y AES. Vigenère aceptará texto introducido por consola o desde un archivo .txt y permitirá mostrar o guardar el resultado en un fichero independiente. AES aceptará ficheros de texto o binarios y conservará los originales.

2.2. JavaFX

El cliente gráfico JavaFX desarrollado con Java 25 LTS para Windows 11 de 64 bits y Linux (sistemas recogidos en el apartado 9) procesará tanto ficheros de texto (.txt) como imágenes (.png), además de admitir otros binarios compatibles

con AES/R2AE. Ofrecerá selección de archivo y destino, cifrado y descifrado, validación, avisos mediante Alert y una vista previa limitada del texto o imagen recuperados cuando proceda. Se mantendrá una base de código común y distribuciones con dependencias nativas adecuadas por sistema.

2.3. Android

La app Android permitirá importar archivos desde almacenamiento local, Bluetooth y funciones nativas de compartir. Tendrá pantallas independientes para descifrado AES de imágenes PNG y descifrado César de archivos .txt. No generará mensajes cifrados nuevos. El entorno de desarrollo será Android Studio Quali 4 (2026.1.4, compilación del 28 de agosto de 2026).

2.4. Unity 6.3

El minijuego 2D Unity 6.3 se distribuirá como ejecutable para Windows 11 de 64 bits (x86-64). Tendrá cinco rondas con dificultad progresiva; el texto y desplazamiento César serán aleatorios dentro de rangos controlados por ronda. Las respuestas se validarán localmente y el enemigo avanzará un tramo por cada fallo, con un sistema explícito de vidas, victoria y derrota.

2.5. Integración y distribución

Python y JavaFX podrán cifrar y descifrar contenedores R2AE compatibles; Android podrá recuperar los que contienen PNG. La interoperabilidad se acreditará con pruebas de intercambio de archivos y recuperación exacta de bytes. Unity implementará y validará localmente César como una aplicación de la misma solución. No habrá dependencias de ejecución entre los cuatro programas.

- Repositorio monorrepositorio en GitHub, con configuración y dependencias documentadas.
- Aplicación Python ejecutable en consola; JavaFX distribuido para Windows/Linux; Android APK; Unity 6.3 compilado y comprobado como ejecutable de Windows 11 x64.
- Sin credenciales incrustadas ni dependencias ocultas; cada aplicación incluirá sus propios módulos y recursos de ejecución.

3. Tratamiento de ficheros | Anexo VI

3.1. Entradas, salidas y almacenamiento (VI.1)

Recurso	Formato y finalidad	Tratamiento previsto
Original	Texto .txt, imagen .png u otro binario	Solo lectura y preservado sin cambios.
Fichero AES	<nombre original>.enc, contenedor R2AE	Resultado independiente; salida provisional .part cuando proceda.
Recuperado	Bytes idénticos al original	Destino elegido o visualización en Android; nunca sobrescritura automática.
Vigenère / César	Texto UTF-8 y .txt	Resultados de texto independientes; origen preservado.
Android / PNG	PNG contenido en un .enc AES	Importación por mecanismos nativos y visualización tras descifrado válido.

Ejemplos: imagen.png → imagen.png.enc; documento.txt → documento.txt.enc; datos.bin → datos.bin.enc. Al descifrar se propondrá un nombre a partir de .enc; si el archivo cambió de nombre, el usuario podrá elegir extensión y destino.

Para recuperar un fichero AES se necesitarán el contenedor .enc y la contraseña de su cifrado. La sal, el nonce, la versión y las iteraciones se guardarán en la cabecera; la contraseña no se incluirá en el contenedor.

3.1.1. Justificación de «.png.enc» frente a «.png» y solicitud de aclaración (4.10)

El apartado 4.10 del pliego menciona la importación y el descifrado de imágenes «.png». El equipo distingue tres estados: imagen.png (fichero PNG original), imagen.png.enc (contenedor R2AE con el contenido original cifrado mediante AES-256-GCM) e imagen.png (imagen recuperada tras verificar la autenticidad del contenedor). La extensión «.enc» indica el carácter cifrado del archivo transferido; «.png» identifica el formato del contenido original y del resultado descifrado.

Justificación técnica: al cifrar íntegramente los bytes de un PNG, los datos resultantes dejan de cumplir la firma, los bloques y las reglas de decodificación de PNG. Renombrar un contenedor R2AE como «.png» no lo convierte en una imagen PNG válida: podría inducir a error a visores, gestores de archivos, filtros MIME o receptores. «.png.enc» evita presentar datos cifrados como imagen legible, mantiene visible la extensión original y permite distinguir el fichero protegido del recuperado sin sobrescribir el original.

Android permitirá seleccionar o recibir «.png.enc» mediante el selector de documentos y los proveedores de contenido compatibles, sin depender exclusivamente del filtro «image/png». Se comprobará la cabecera y la autenticidad de R2AE; solo después se verificarán el contenido PNG recuperado y su decodificación para mostrarlo en pantalla. El fichero recibido no se modificará, y los errores se comunicarán mediante avisos nativos.

Solicitud de aclaración contractual: se pide al Departamento de IT confirmar si la referencia a archivos «.png» en el apartado 4.10 designa el tipo de imagen que debe poder recuperarse y visualizarse, admitiendo el transporte cifrado como «.png.enc». Esta propuesta se presenta como equivalencia funcional para revisión, no como modificación aprobada unilateralmente. Si se exigiera que el propio archivo cifrado tuviera extensión «.png» y fuera un PNG válido, se definirá de común acuerdo un encapsulado PNG que contuviera los datos cifrados, con las pruebas pertinentes; cambiar únicamente la extensión de R2AE no sería una solución técnicamente equivalente. La opción definitiva quedará sujeta a confirmación por escrito antes de su aceptación.

3.2. Recorrido de la información (VI.2)

El mismo proceso lógico se aplicará a Python y JavaFX, y al descifrado AES en Android. Cada aplicación ejecutará sus operaciones localmente conforme a la especificación R2AE compartida.

Paso	Cifrado AES	Descifrado AES
1. Entrada	Seleccionar original y ruta de salida.	Seleccionar contenedor .enc.
2. Validación	Comprobar acceso, destino y recursos.	Comprobar cabecera R2AE y pedir contraseña.
3. Criptografía	Generar contraseña, sal y nonce; derivar clave; AES-GCM.	Derivar clave con parámetros del contenedor; AES-GCM.
4. Integridad	Incluir cabecera autenticada y etiqueta GCM.	Verificar etiqueta antes de publicar contenido.
5. Resultado	Guardar temporal y publicar .enc nuevo.	Guardar recuperado o mostrar PNG; sin sobrescritura.

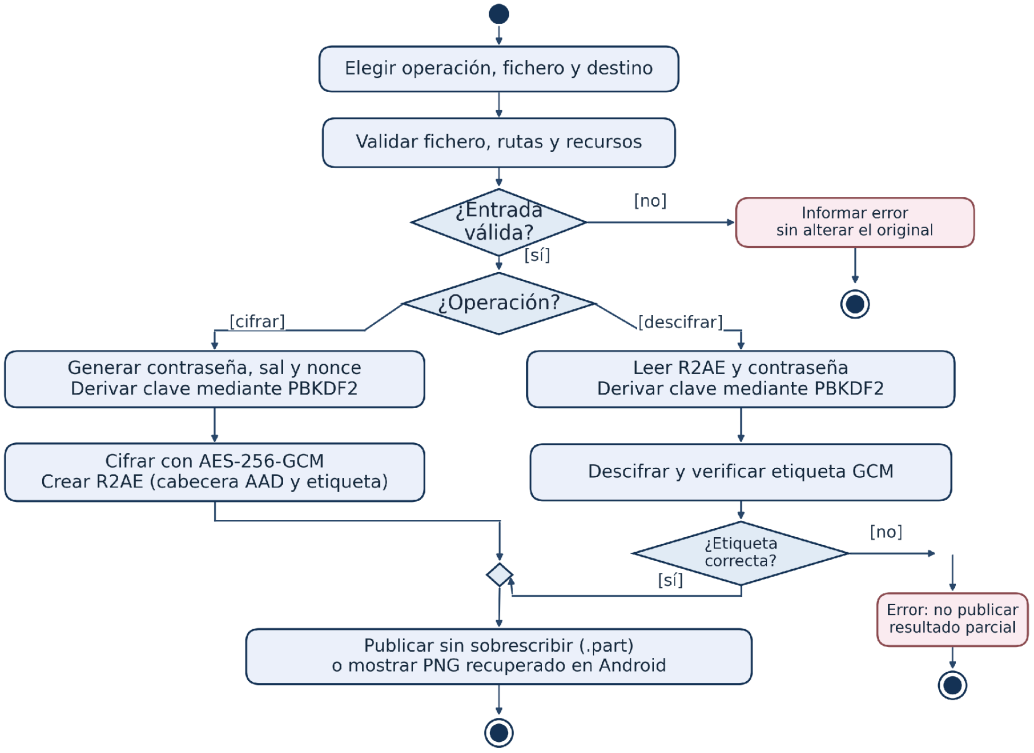


Figura UML 1. Diagrama de actividades UML del cifrado/descifrado AES y publicación segura del resultado.

Notación UML: nodo inicial y final, acciones, decisiones y guardas. Los bocetos visuales del apartado 5 son propuestas de interfaz y no se presentan como diagramas UML.

Vigenère y César trabajan sobre texto. En Python, Vigenère transforma texto introducido o leído desde .txt y permite guardar una salida independiente; Android recupera órdenes César desde .txt y Unity genera/valida desafíos César localmente.

3.3. Decisión técnica: AES-256-GCM y contraseñas (VI.2)

Se adopta AES-256-GCM por incorporar cifrado y autenticación de datos. La clave de 256 bits se derivará mediante PBKDF2-HMAC-SHA256, con 200.000 iteraciones y sal aleatoria de 16 bytes por fichero. Cada cifrado utilizará un nonce aleatorio nuevo de 12 bytes y etiqueta GCM de 16 bytes.

Campo de R2AE	Longitud / codificación
Identificador ASCII «R2AE»	4 bytes
Versión	1 byte; valor inicial 1
Iteraciones PBKDF2	4 bytes, entero sin signo big-endian
Sal	16 bytes aleatorios
Nonce GCM	12 bytes aleatorios
Contenido cifrado	Longitud variable, igual a la del fichero original
Etiqueta GCM	16 bytes, situada al final

La cabecera ocupa 37 bytes y se autentica íntegramente como AAD. La sobrecarga del contenedor R2AE es de 53 bytes. Python, JavaFX y Android compartirán la misma estructura, codificación de parámetros, tratamiento de contraseña y vectores de prueba.

Gestión de contraseñas: en cada operación de cifrado, Python y JavaFX generarán automáticamente una contraseña alfanumérica de 16 caracteres seleccionados de A–Z, a–z y 0–9 mediante generadores criptográficamente seguros

(secrets en Python y SecureRandom en Java). No se permitirá elegir manualmente una contraseña nueva al cifrar. La contraseña generada se mostrará para que pueda guardarse; al descifrar deberá introducirse la correspondiente. Android solo la solicitará para descifrar.

Las contraseñas no se guardarán en el archivo R2AE, en el código, en logs ni en configuraciones. El destinatario recibirá la contraseña por un canal separado y seguro. Perderla impedirá el descifrado ordinario. La contraseña de 16 caracteres no equivale a una clave de 128 bits: PBKDF2 deriva una clave AES de 256 bits a partir de su entropía original.

3.4. Tamaños, incidencias y protección (VI.3)

Tamaño	Comportamiento previsto
0 bytes	Cifrado válido: contenedor de 53 bytes y recuperación vacía.
1 byte	Contenedor de 54 bytes y recuperación idéntica.
1 MiB y ≥20 MiB	Cifrar, descifrar y comparar bytes y SHA-256.
Por encima de los tamaños probados	Sujeto a disponibilidad de recursos y límites finales documentados.

Conforme al apartado 4.5 del pliego, la solución final deberá procesar mediante AES ficheros de 0 bytes, 1 byte, 1 MiB y al menos 20 MiB, recuperando los bytes originales y evitando modificaciones o sobrescrituras indebidas. Se prevé comenzar con un tratamiento en memoria para los tamaños que permitan los recursos de cada plataforma; si se detectan necesidades de memoria o espacio, se estudiará el uso de procesamiento incremental o ficheros temporales seguros, sin comprometer la integridad ni publicar resultados incompletos. No se fija todavía un tamaño máximo superior. La capacidad real y los límites definitivos se determinarán con las pruebas de la solución final.

Incidencia	Respuesta prevista
Clave incorrecta o datos alterados	Fallo de autenticación; no publicar resultado parcial.
Fichero inexistente, corrupto o sin permisos	Error comprensible en CLI, Alert JavaFX o aviso nativo Android.
Destino ya existente	Elegir otra ruta o nombre; prohibida sobrescritura automática.
Disco lleno, fallo de escritura o recursos insuficientes	Cancelar, liberar recursos, limpiar .part cuando sea posible y conservar originales.

3.4.1. Estrategia prevista por tamaños y limitaciones conocidas

Para comprobar la aceptación se prevén pruebas reproducibles con archivos de 0 B, 1 B, 1 MiB y al menos 20 MiB, incluyendo texto y datos binarios. Se verificará que el descifrado recupera exactamente los bytes originales, contrastando tamaño, contenido y SHA-256; se controlarán también las situaciones de falta de memoria, espacio insuficiente, errores de acceso y destino existente. En Android se verificará la recuperación y visualización de imágenes PNG cifradas compatibles. Las limitaciones conocidas por ahora dependen de la memoria disponible, el almacenamiento libre y los permisos de cada dispositivo; aún no se dispone de mediciones ni de un límite máximo empíricamente determinado. Estas comprobaciones se ejecutarán durante el desarrollo y la validación final, una vez aprobado el contrato.

Los originales se abrirán en lectura. Cuando el sistema lo permita se utilizará fichero temporal .part, seguido de publicación sin reemplazo de otro archivo existente. Se gestionarán excepciones y cierre seguro de recursos.

3.5. Plan de comprobación (VI.4)

Las pruebas siguientes están previstas y sus resultados se acreditarán en la entrega final. Se diferencian pruebas unitarias, funcionales, de integración y de sistema/aceptación, conservando sus identificadores y criterios. La automatización de pruebas en CI/CD se limitará a Python; las comprobaciones de JavaFX, Android y Unity se ejecutarán y documentarán localmente.

3.5.1. Pruebas unitarias (lógica aislada)

ID	Prueba prevista	Resultado esperado
P-01	Vigenère con signos, espacios, mayúsculas, ñ y tildes.	Texto reversible según las reglas documentadas.
P-02	AES con 0 y 1 byte.	Recuperación exacta, incluso vacío.
P-11	Generación de contraseñas.	16 caracteres alfanuméricos con generador criptográfico seguro.

3.5.2. Pruebas funcionales (interacción y errores)

ID	Prueba prevista	Resultado esperado
P-04	Contraseña errónea y contenedor alterado.	Error controlado sin salida publicada.
P-05	Error de escritura, permisos y destino existente.	Original preservado y no sobrescritura automática.
P-08	Android: PNG AES y .txt César; errores.	Contenido recuperado y avisos sin cierre inesperado.
P-09	Unity: respuestas, enemigo, puerta y estados.	Partida, victoria y derrota verificables.
P-13	Contraseña perdida o incorrecta.	Sin recuperación prevista sin la clave válida; fallo seguro.

3.5.3. Pruebas de integración (interoperabilidad)

ID	Prueba prevista	Resultado esperado
P-06	Ficheros R2AE cruzados Python/JavaFX/Android.	Interoperabilidad y visualización de PNG en Android.
P-12	Descifrar con contraseña generada por otra app.	Mismos bytes con clave correcta.

3.5.4. Pruebas de sistema y aceptación (ejecución completa)

ID	Prueba prevista	Resultado esperado
P-03	AES con 1 MiB y ≥20 MiB.	SHA-256 coincidente y comparación exacta de bytes.
P-07	Fatjar JavaFX Windows y Linux.	Operaciones AES y alertas funcionales.
P-10	CI/CD Python y cobertura de las aplicaciones.	Solo la CI de Python ejecuta tests y falla bajo el 80 %; el resto acredita su cobertura y pruebas fuera de CI/CD.

En las pruebas AES se contrastarán tamaño, SHA-256 y contenido byte a byte del original frente al recuperado. La comparación requiere conservar el original como referencia durante la prueba.

Método de ejecución y evidencia: las pruebas unitarias validarán funciones aisladas y servirán para medir la cobertura ≥80 % de cada aplicación. Las funcionales comprobarán los flujos de usuario y la conservación de archivos; las de integración verificarán R2AE entre clientes; y las de sistema y aceptación comprobarán tamaños y distribución. Únicamente las pruebas de Python se automatizarán en CI/CD. Las demás se ejecutarán localmente y aportarán sus informes y evidencias, sin confundirse con pruebas unitarias.

4. Componentes y comportamiento funcional

4.1. Aplicación Python (4.3)

Se desarrollará una aplicación Python 3.14 por consola mediante menú interactivo con opciones para cifrar y descifrar Vigenère, cifrar y descifrar AES y salir. Los errores controlados permitirán regresar al menú, sin finalizar inesperadamente.

Vigenère aceptará mensajes por consola o desde .txt. Se utilizará el alfabeto A–Z, preservando mayúsculas y minúsculas; espacios, signos, dígitos, ñ y tildes permanecerán sin cifrar y no avanzarán la clave. Se aceptarán claves de entre 1 y 64 letras A–Z. Los resultados podrán visualizarse o guardarse en .txt independiente.

AES permitirá seleccionar origen y destino para ficheros de texto o binarios, generará automáticamente contraseña de 16 caracteres al cifrar y solicitará la contraseña original al descifrar. Implementará R2AE, protección del original y errores de acceso, escritura y autenticación. La lógica estará separada del menú para facilitar los tests unitarios.

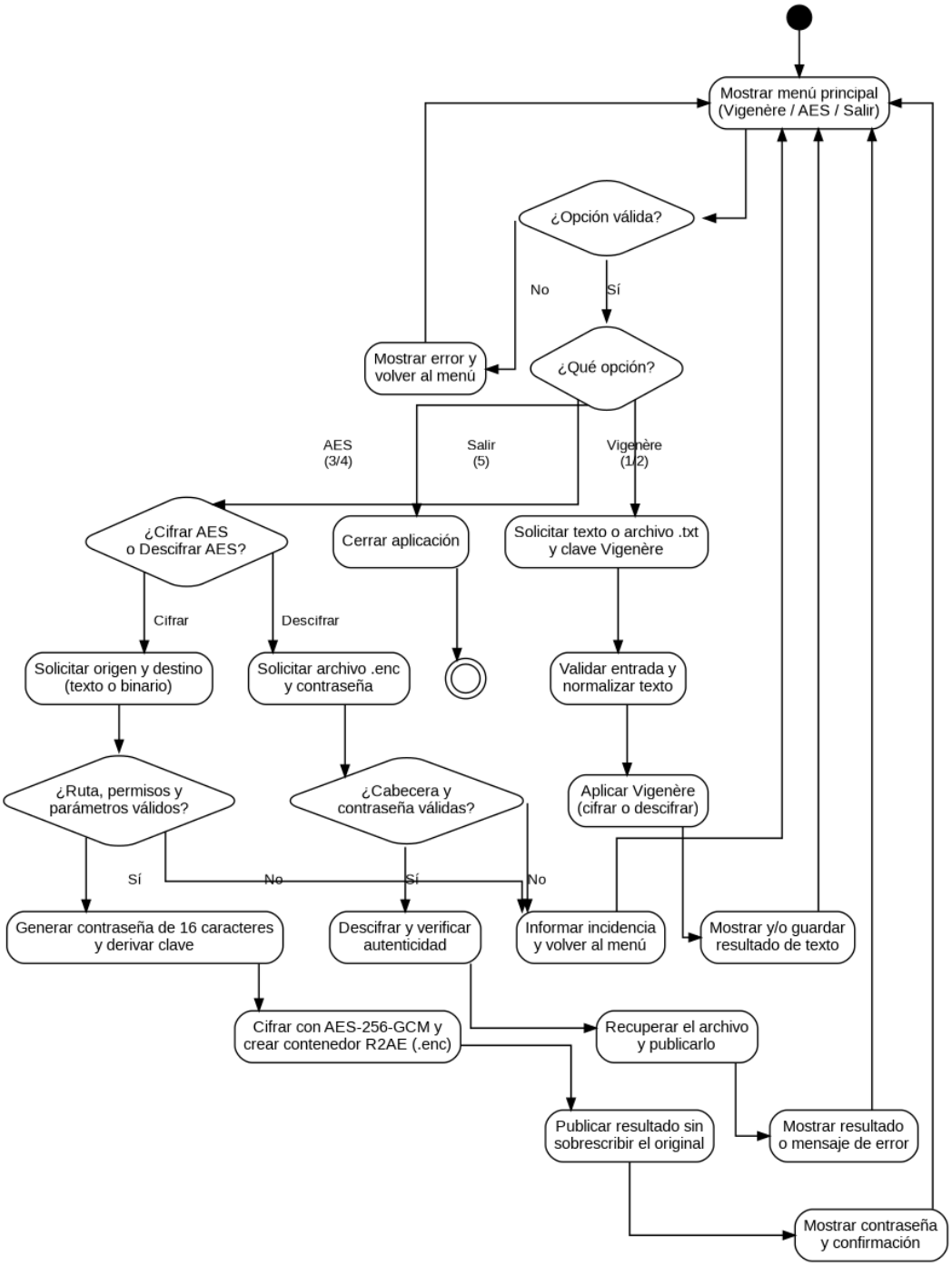


Figura UML 3. Diagrama de actividades UML de la aplicación Python CLI.

Notación UML: el menú principal decide entre operaciones Vigenère, AES y salida; las rutas inválidas regresan al menú con mensaje de error.

4.2. Cliente JavaFX (4.8)

El cliente gráfico JavaFX sobre Java 25 LTS operará tanto sobre archivos de texto (.txt) como sobre imágenes (.png), además de aceptar otros ficheros binarios. Permitirá elegir origen y destino, cifrar/descifrar mediante AES-256-GCM, ver la contraseña de 16 caracteres generada automáticamente, introducirla para descifrar y verificar los bytes cuando se disponga del original. En texto e imágenes recuperados podrá mostrar una vista previa con límites de tamaño y decodificación; el tratamiento criptográfico preservará siempre los bytes, sin transformar el contenido antes de cifrar. Ningún resultado sobrescribirá automáticamente el original ni un destino preexistente.

Se utilizarán Alert para errores y confirmaciones, accesibilidad mediante etiquetas asociadas, navegación por teclado y mensajes comprensibles; se aplicará cierre seguro de recursos con try-with-resources o finally cuando corresponda. La misma base de código producirá distribuciones compatibles con Windows 11 x64 y Linux, incluyendo dependencias JavaFX nativas apropiadas.

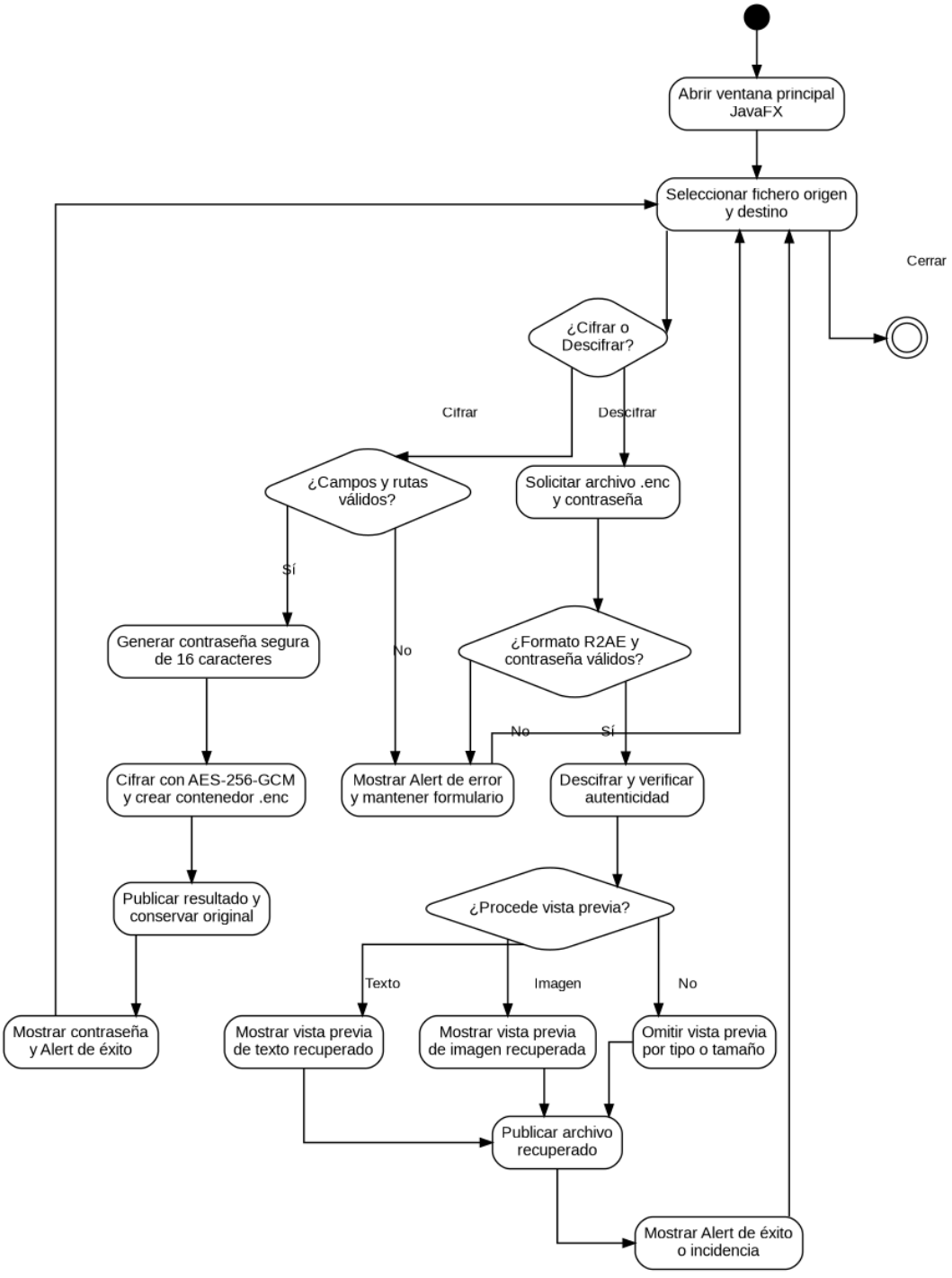


Figura UML 4. Diagrama de actividades UML del cliente JavaFX para operaciones AES, vista previa y alertas.

Notación UML: el cliente cifra y descifra contenedores R2AE, utiliza Alert para incidencias y muestra vista previa del contenido recuperado cuando procede.

4.3. Aplicación Android (4.10)

En rol Auror, la aplicación será exclusivamente de consulta y descifrado, con pantalla principal y dos pantallas funcionales independientes. Usara documentos .txt y contenedores AES de imágenes .png mediante almacenamiento local se usara ACTION_OPEN_DOCUMENT y se recibirá mediante proveedores compatibles (incluido intercambio por Bluetooth o red).

Pantalla AES: solicitará la contraseña generada previamente por Python o JavaFX y mostrará la imagen PNG recuperada sin sobrescribir el archivo recibido. Pantalla César: abrirá .txt, solicitará el número de desplazamiento y mostrará el texto descifrado.

Los errores de clave, formato, desplazamiento o corrupción se notificarán con ventanas emergentes sin cerrar inesperadamente la aplicación. Android no cifrará mensajes nuevos ni generará contraseñas para nuevos cifrados.

4.4. Minijuego Unity 6.3 (4.11)

Se implementará un minijuego 2D con un dementor representado por un prefab enemigo, una puerta objetivo y cinco rondas de mensajes César. En cada ronda se tomará una frase del banco correspondiente y un desplazamiento seudorrandom del rango permitido. Se mostrará el texto cifrado y el desplazamiento necesario para recuperarlo, sin revelar la solución; el jugador escribirá el texto descifrado. La aleatoriedad no altera el nivel de dificultad asignado a cada ronda.

La escena inicia en Tutorial; después comienza En partida con 3 vidas y la ronda 1/5. Cada error válido desplaza automáticamente al enemigo una de cuatro posiciones discretas hacia la puerta; al cuarto error de una misma tentativa alcanza la puerta, se pierde exactamente una vida, se destruye el enemigo actual y se instancia un nuevo prefab en el inicio para repetir el mismo desafío. Un acierto elimina el enemigo y genera la siguiente ronda con un nuevo prefab y el contador de avances a cero. Al superar cinco rondas aparece Victoria; con cero vidas aparece Derrota. Los botones «Reiniciar» en ambas pantallas llaman a SceneManager.LoadScene para restaurar la escena desde cero; «Salir» cierra la aplicación. Se detienen movimiento y validación al terminar.

Físicas2D: el enemigo incorporará Rigidbody2D de tipo Kinematic y Collider2D; se moverá por tramos mediante Rigidbody2D.MovePosition en FixedUpdate. La puerta tendrá Collider2D con isTrigger activado; OnTriggerEnter2D detectará la llegada y aplicará una sola pérdida de vida mediante protección frente a eventos repetidos. No se usará OnCollisionEnter2D para esta puerta, puesto que el enemigo la atraviesa; solo se utilizaría ese evento si se decidiese una puerta con colisión física sólida. Las acciones de ronda, normalización y estado se separarán de la vista para facilitar Edit Mode Tests.

4.4.1. Progresión, normalización y apoyo al jugador

Ronda	Complejidad controlada	Reglas
1	Tutorial: 4–6 letras; desplazamientos 1–3	Ejemplo guiado, indicación de cómo descifrar y pista visible
2	Palabras o frases cortas; desplazamientos 1–5	Pista opcional; se mantiene el mismo número de vidas
3	Frases 8–12 letras; desplazamientos 1–10	Menos apoyo inicial; pista opcional
4	Frases 12–18 letras; desplazamientos 1–17	Mayor longitud y más posibles desplazamientos
5	Frases 18–25 letras; desplazamientos 1–25	Desafío final; pistas voluntarias sin revelar solución

Regla de errores: cada cuatro respuestas incorrectas de un mismo intento causan una pérdida de vida; se dispone de tres vidas en toda la partida, por lo que 12 errores acumulados en intentos fallidos de puerta producen derrota si no se han superado antes las cinco rondas. Un acierto reinicia el avance del enemigo; no recupera vidas. La semilla o secuencia de selección podrá fijarse en tests para reproducir rondas.

Validación: quitar espacios al principio y al final (trim), convertir a mayúsculas de forma estable, normalizar espacios consecutivos y admitir equivalencia de vocales acentuadas sin eliminar ni transformar la Ñ (NIÑO no equivale a NINO). El banco inicial contendrá frases A–Z y espacios, sin signos ni tildes; el algoritmo César desplaza exclusivamente A–Z, preserva espacios y cualquier carácter fuera de ese alfabeto. Se explicarán estas reglas al jugador. Las entradas vacías o solo espacios recibirán aviso y no contarán como error de avance. Se probarán casos con mayúsculas, espacios, vocales acentuadas, Ñ y entradas vacías.

Tutorial: pantalla inicial breve con un ejemplo de César, desplazamiento mostrado, respuesta guiada y controles explicados. Pistas opcionales mostrarán cómo desplazar una letra, sin revelar la frase completa. La escena incluirá pausa y un indicador textual y visual de vidas, ronda y avance restante hasta la puerta.

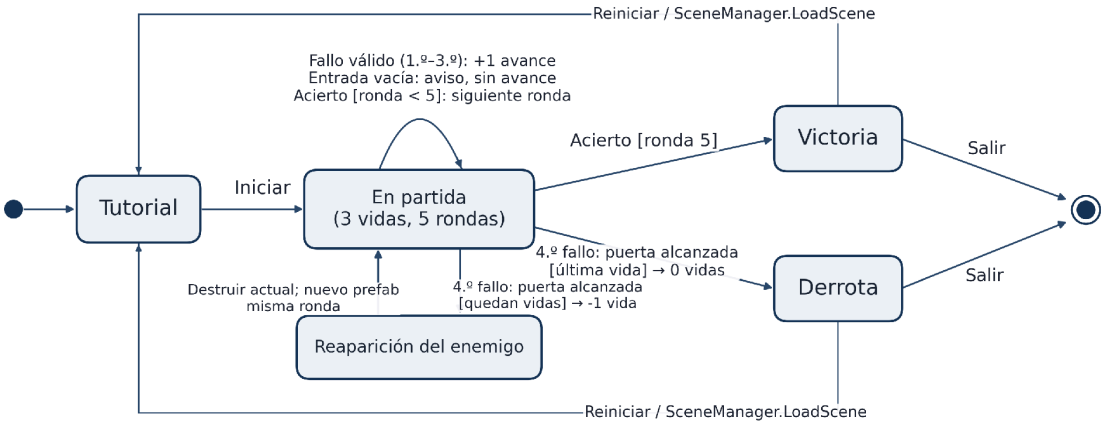


Figura UML 2. Diagrama de máquina de estados UML del minijuego; las transiciones indican eventos y condiciones.

5. Bocetos funcionales

Los siguientes bocetos representan de forma preliminar el flujo de usuario y las operaciones principales de los cuatro componentes. Las ilustraciones no constituyen capturas de una implementación definitiva.

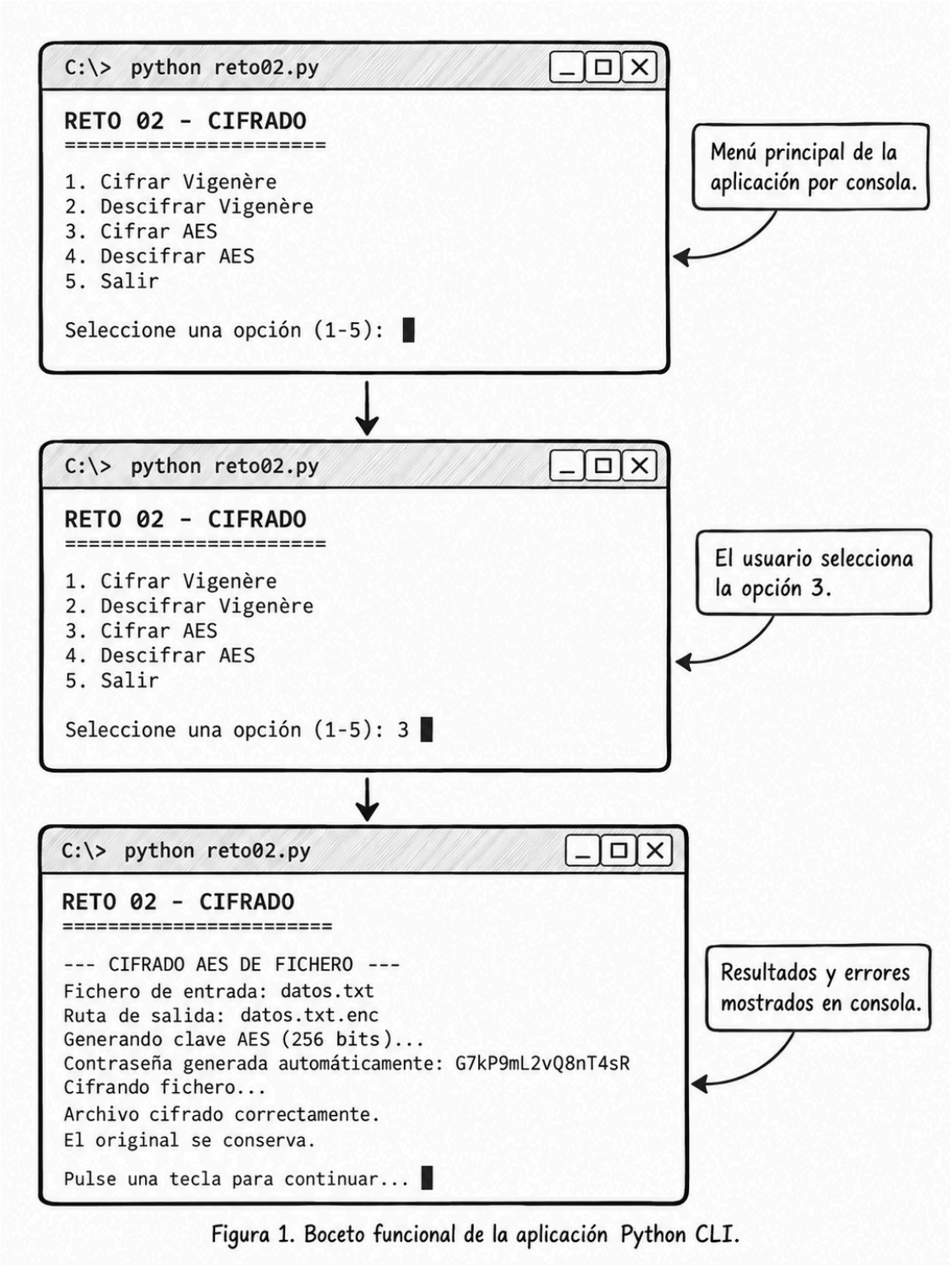


Figura 1. Boceto funcional de la aplicación Python CLI.

El menú contempla Vigenère y AES. En la implementación efectiva; las contraseñas se generarán automáticamente con 16 caracteres.

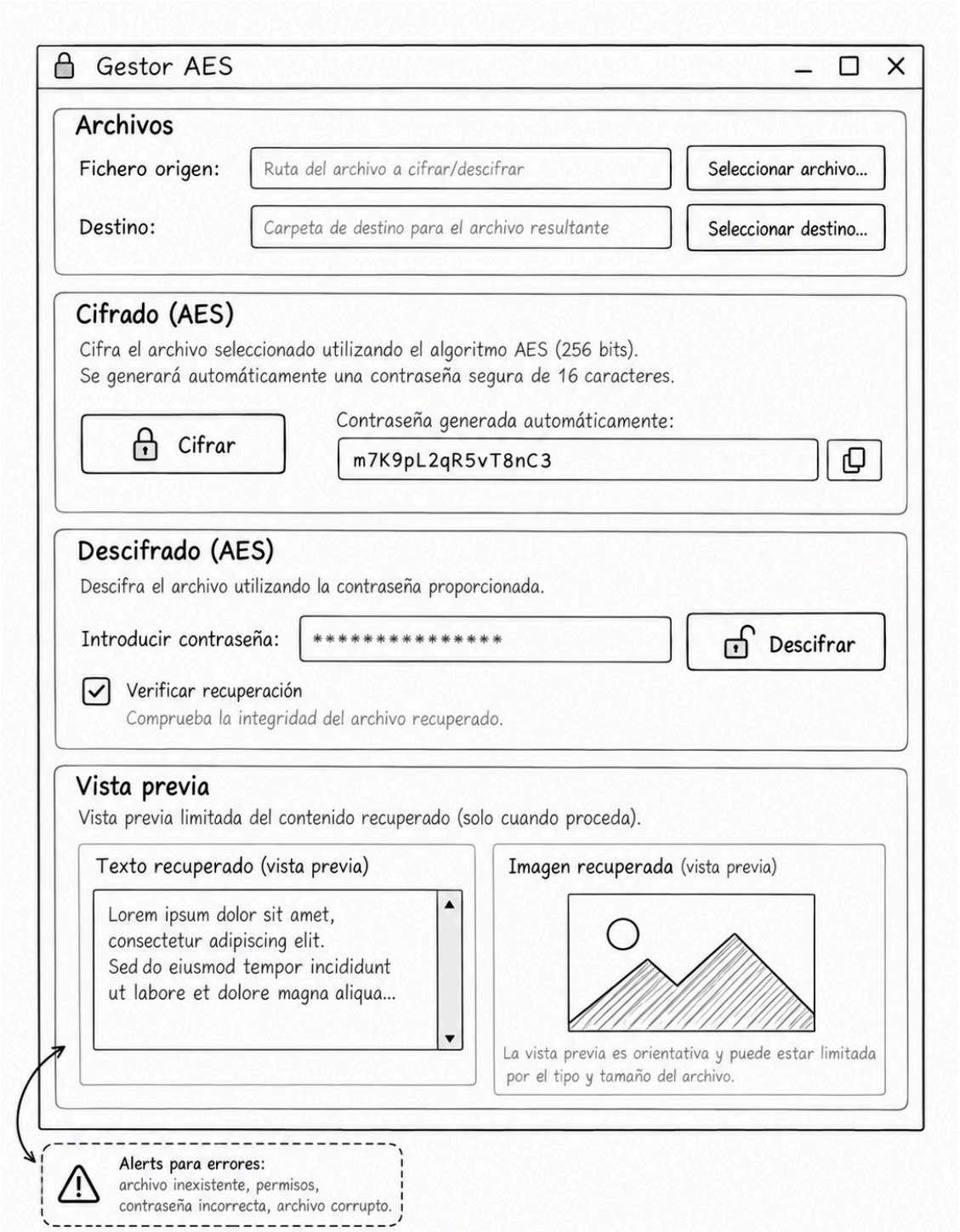


Figura 2. Boceto funcional del cliente JavaFX para operaciones AES.

El campo de contraseña generada se utilizará al cifrar; para descifrar se introducirá la contraseña del archivo recibido. La verificación byte a byte requiere un fichero original de referencia.

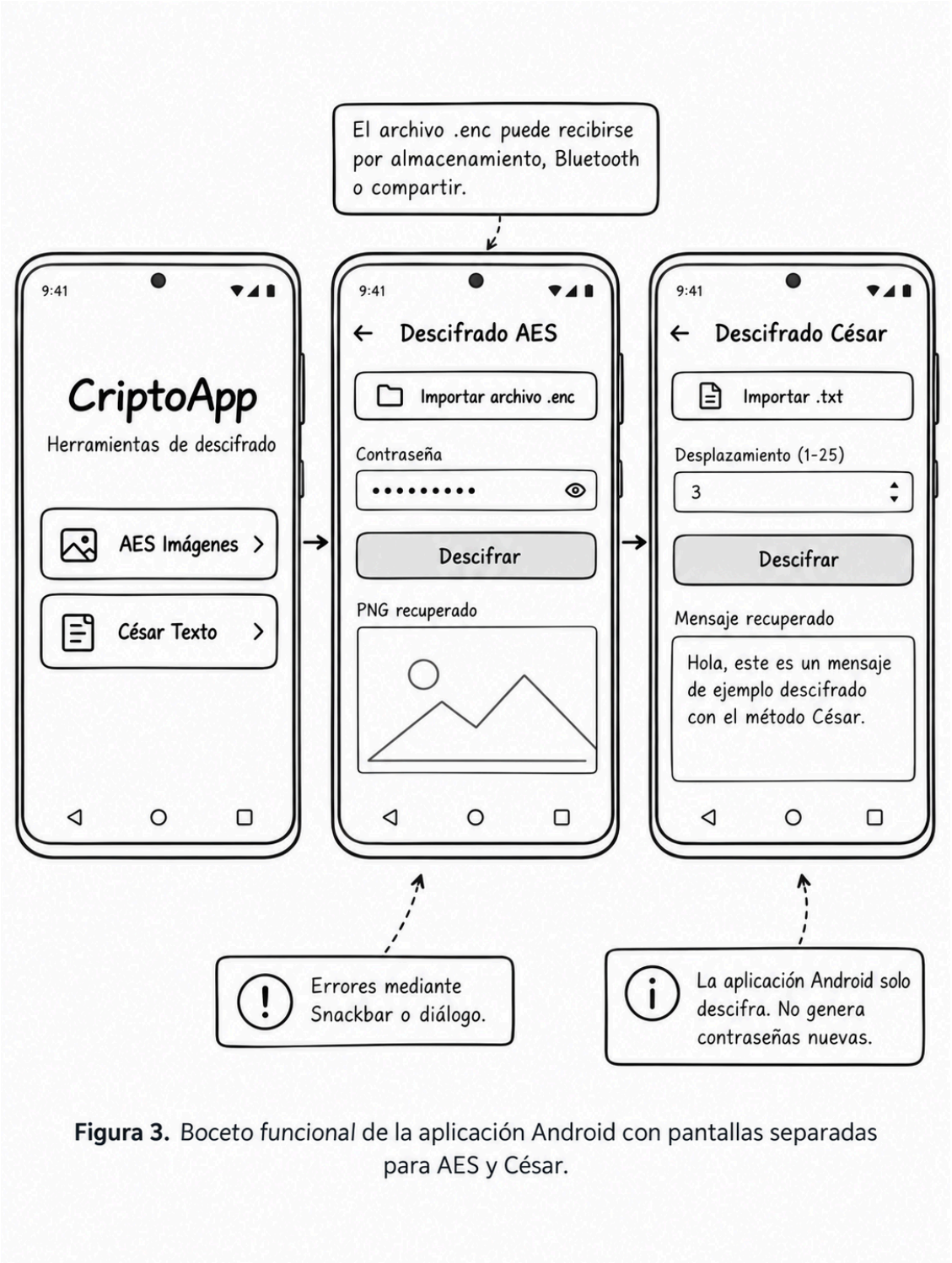


Figura 3. Boceto *funcional* de la aplicación Android con pantallas separadas para AES y César.

Android solo descifra: recibe contenedores .enc con imágenes PNG y archivos .txt con cifrado César.

Esta app no genera contraseñas.



Figura 4. Boceto funcional del minijuego Unity 2D con puerta, enemigo, validación y estados.

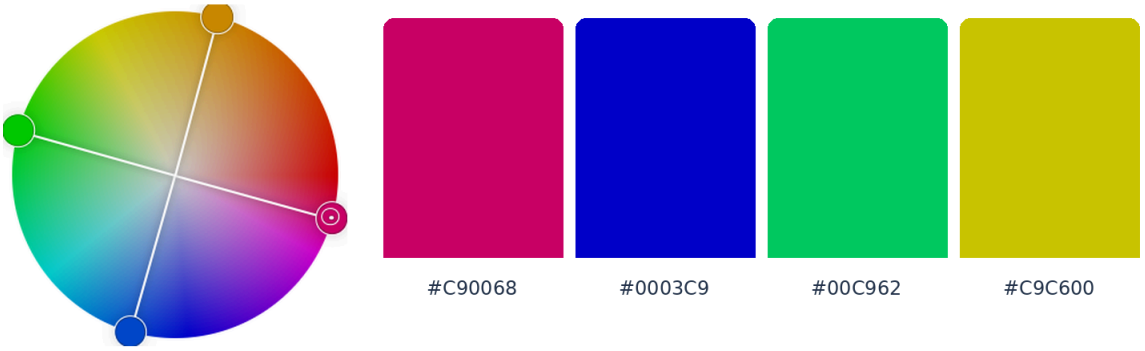
El contacto con la puerta restará una vida; se producirá derrota cuando se agoten las vidas. El personaje definitivo podrá ser un dementor o un búho. Las decisiones de arte y movimiento podrán evolucionar.

5.1. Carácter provisional de los bocetos

Los bocetos son preliminares y orientativos. Podrán ser refinados, modificados o sustituidos durante el desarrollo; el producto final podrá variar en apariencia visual, distribución, navegación y experiencia de usuario. Estos cambios buscarán mejorar la usabilidad, accesibilidad y calidad, pero no podrán eliminar ni incumplir los requisitos funcionales y técnicos del pliego. No se exige reproducir exactamente las ilustraciones.

5.2. Paleta cromática e identidad visual

Se propone una paleta tetrádica de cuatro colores como referencia de identidad visual para las interfaces de JavaFX, Android y Unity. Su aplicación permitirá mantener una coherencia estética entre las distintas aplicaciones.



Los tonos propuestos son magenta (#C90068), azul (#0003C9), verde (#00C962) y amarillo (#C9C600). Se emplearán como acentos, indicadores o elementos identificativos según el contexto; para superficies extensas, texto y controles se priorizará la legibilidad, el contraste y la accesibilidad.

5.3. Tipografía y accesibilidad (A11Y, WAI-ARIA y equivalencias nativas)

Tipografía propuesta: Noto Sans para títulos y cuerpo, con alternativas de sistema equivalentes cuando no esté instalada (Arial o DejaVu Sans en escritorio; Roboto en Android y fuente legible integrada en Unity). Se mantendrán jerarquía visual, tamaño mínimo funcional y escalabilidad. La tipografía concreta y sus licencias se comprobarán antes de distribuir activos; la ausencia de una fuente no bloqueará la lectura por fuentes de sustitución.

Criterios A11Y: navegación por teclado y foco visible en JavaFX, etiquetas accesibles en campos y botones, orden de tabulación coherente, tamaños de interacción adecuados, feedback no dependiente únicamente del color y contrastes objetivo WCAG 2.2 AA (4,5:1 para texto normal y 3:1 para texto grande o componentes esenciales). Se revisarán mensajes de error, alternativas textuales para imágenes relevantes y reducción de animaciones molestas cuando sea viable.

ARIA designa atributos semánticos para interfaces web; las aplicaciones de este reto son nativas y no se aplicarán atributos HTML aria-* donde no existe DOM. Se documentarán los principios equivalentes: accessibleText / accessibleRole y asociación de labels en JavaFX; contentDescription, labels y foco de accesibilidad en Android; TextMeshPro, controles por teclado y texto alternativo de instrucciones y estados en Unity. Si en el futuro se presenta contenido web, se usarán HTML semántico y WAI-ARIA solo cuando sea necesario.

6. Repositorio, automatización y calidad

GitHub será el medio habitual de intercambio y control de versiones del proyecto, alojado en un único monorrepositorio. Cada integrante usará su propia identidad Git. Se conservarán evidencias de autoría, ramas y revisiones, así como de la CI/CD de Python y de las validaciones locales de los demás clientes.

6.1. Estructura replicable (4.13)

Directorio	Contenido
python/	CLI, Vigenère, AES y pruebas unitarias.
javafx/	Escritorio JavaFX, AES, empaquetado y pruebas locales.
android/	Aplicación Kotlin y pruebas locales.
unity/	Proyecto Unity 6.3 y pruebas locales.
tests/	Pruebas comunes de integración e interoperabilidad.
docs/	Manual de instalación, uso, decisiones y evidencias.
.github/workflows/	Workflow de GitHub Actions exclusivamente para Python.

README y configuraciones reproducibles documentarán las dependencias, la compilación, las pruebas y las instrucciones de ejecución de cada aplicación de forma independiente. No se requiere desplegar infraestructura central para utilizarlas.

6.2. Estrategia Git y trazabilidad (4.7)

Se utilizarán main (estable), develop (integración) y ramas para cada aplicación. No habrá push directo a main ni develop; los cambios se integrarán mediante Pull Requests tras revisión y validación.

- Al menos tres commits sustanciales, con mensajes descriptivos, por cada integrante durante las tres semanas.
- Al menos dos ramas de desarrollo efectivamente utilizadas además de main.
- Al menos una Pull Request integrada con descripción y revisión o comentario de otro integrante.
- Trazabilidad de autores, fechas, contenido y participación, evitando cambios acumulados sin explicación.

6.3. Hooks y CI/CD (4.7 y 4.13)

Los hooks pre-commit comprobarán formato, lint y pruebas rápidas de Python. GitHub Actions ejecutará en push y Pull Request un único workflow de pruebas y cobertura para Python, con fallo si no supera las comprobaciones o la cobertura mínima del 80 %. No se comprometen jobs ni pipelines CI/CD de pruebas para JavaFX, Android o Unity: sus validaciones se efectuarán localmente y se documentarán. La protección de ramas exigirá el resultado correcto del workflow Python. Se solicita la validación de este alcance por el profesorado.

6.4. Cobertura mínima del 80 % por aplicación (4.12)

Se mantiene como objetivo contractual una cobertura unitaria ≥80 % del código propio medible en cada aplicación: Python, JavaFX, Android y Unity 6.3, sin promediar porcentajes. Solo Python tendrá comprobación automática y bloqueo por cobertura en GitHub Actions. Para las otras aplicaciones se obtendrán informes de cobertura mediante ejecuciones locales, sin jobs ni umbrales de fallo en CI/CD. Se justificarán las exclusiones y el método de medición.

Aplicación	Herramientas previstas	Áreas comprobadas
Python	pytest / pytest-cov	Vigenère, AES, contraseñas, archivos y errores.
JavaFX	JUnit / JaCoCo	AES, R2AE, validación y archivos.
Android	JUnit + cobertura JVM/Kotlin	AES, César, selección y errores.
Unity	Unity Test Framework + Code Coverage	César, rondas, respuestas y estados.

La cobertura del 80 % corresponderá a pruebas unitarias y no sustituirá pruebas funcionales, de integración, Play Mode ni demostraciones manuales. Python utilizará pytest --cov-fail-under=80 en CI/CD. Para JavaFX, Android y Unity se prevén JUnit/JaCoCo, cobertura Kotlin/JVM y Unity Test Framework/Code Coverage en ejecución local.

6.5. Validación local y evidencias

Se preparará al menos una ejecución local con act del workflow Python en una máquina con Docker disponible, comparándola con GitHub Actions. act no sustituye el resultado remoto; las dificultades de ejecución se documentarán y justificarán. Se conservarán logs, cobertura, tests, formato y lint del pipeline Python; de JavaFX, Android y Unity se aportarán informes de pruebas locales, evidencias funcionales, empaquetados y capturas de las validaciones finales.

6.6. Python: pyproject.toml y verificación demostrable

El archivo python/pyproject.toml centralizará metadatos, dependencias directas y de desarrollo, configuración de pytest (ubicación tests/), pytest-cov/coverage (código fuente y exclusiones justificadas) y ruff (formato y reglas de lint). El workflow Python ejecutará, en orden, instalación reproducible, ruff format --check ., ruff check ., pytest con informe XML/HTML y --cov-fail-under=80; después publicará artefactos e informe. En la defensa se mostrará una ejecución completa del pipeline y se justificarán herramientas enseñadas en clase que no se hayan adoptado.

7. Anexo de planificación ágil | Discovery y dos sprints

La guía de trabajo se organizará en tres semanas: semana 1, Discovery; semanas 2 y 3, Sprint 1 y Sprint 2 respectivamente. Las cuatro aplicaciones pueden progresar en paralelo durante los sprints, pero Discovery antecede a los dos incrementos de desarrollo. Las asignaciones y horas siguientes son una propuesta de capacidad, revisable por el equipo antes de entregar el contrato; no se presentan como horas ya realizadas.

7.1. Equipo, capacidad y roles propuestos

Integrante	Responsabilidad propuesta	Discovery	Sprint 1	Sprint 2	Total
Carlos	JavaFX, experiencia de usuario, pruebas y accesibilidad	16 h	16 h	16 h	48 h
Koldo	Python, interoperabilidad R2AE, diseño y documentación	16 h	16 h	16 h	48 h
Kenneth	Android, Unity y validación técnica/CI	16 h	16 h	16 h	48 h
Total	Capacidad orientativa; 16 h/persona/semana	48 h	48 h	48 h	144 h

Si cambia la capacidad, se recalcularán horas y alcance antes de iniciar los sprints. Roles Scrum propuestos: Carlos facilita las ceremonias (Scrum Master rotativo), Koldo centraliza dudas de requisitos y prioridades (interlocutor de Product Owner, sin sustituir al cliente), y los tres actúan como desarrolladores y revisores. La autoría individual se registra por Git y por Issue.

7.2. Tablero, ceremonias y criterios de aceptación

Tablero de GitHub Projects (vista Backlog): Current iteration · Projecto_Enigma . Según la captura facilitada el 28/10/2026, la Issue #1 «Aprobación de la propuesta» está en «In progress» y la Issue #2 «Creación estructura de proyecto (ramas y commits iniciales)» en «Backlog», ambas con prioridad «Urgent». Las tablas de las secciones 7.3 a 7.5 son una posible organización del trabajo: sus identificadores D-01/S1-01/S2-01 y asignaciones no acreditan la creación de esas Issues ni fijan responsabilidades definitivas.

El tablero dispone de columnas Backlog, Ready, In progress, In review y Done. El equipo revisará y concretará las tareas que se incorporen como Issues, con prioridades, responsables, estimaciones en horas-persona, criterios de aceptación y enlaces a PR o al workflow de Python cuando corresponda. Se mantendrán Daily Scrum de máximo 15 minutos en días lectivos durante la primera hora de alguna asignatura ETHAZI, Sprint Planning al comienzo de Sprint 1 y Sprint 2, y Review y Retrospective al terminar cada sprint. Discovery concluirá con la revisión de riesgos, viabilidad de 20 MiB, arquitectura acordada y backlog propuesto revisado.

7.3. Backlog propuesto para Discovery (semana 1; 48 h)

ID	Prior.	Resp.	h-pers.	Criterio de aceptación
D-01	Alta	Carlos	10	Monorrepo, ramas y README inicial; estructura clonable y revisada
D-02	Alta	Koldo	10	R2AE, decisiones de arquitectura y diagramas UML revisables
D-03	Alta	Kenneth	10	Ensayo 20 MiB y registro de tiempo, memoria e integridad o riesgo documentado
D-04	Media	Carlos	6	Bocetos, tipografía y matriz A11Y por plataforma
D-05	Alta	Koldo	6	Backlog, capacidad y riesgos documentados como propuesta inicial, con estimaciones y criterios revisables
D-06	Alta	Kenneth	6	Primer workflow Python, pyproject.toml y validación de ejecución CI

7.4. Backlog propuesto para Sprint 1 (semana 2; 48 h)

ID	Prior.	Resp.	h-pers.	Criterio de aceptación
S1-01	Alta	Koldo	10	CLI y Vigenère funcionales con tests para entradas y errores
S1-02	Alta	Koldo	6	AES/R2AE Python genera/recupera bytes y contraseña segura
S1-03	Alta	Carlos	10	JavaFX cifra/descifra .txt y .png con selección de origen/destino
S1-04	Alta	Carlos	6	JavaFX incluye avisos, teclado y vista previa controlada
S1-05	Alta	Kenneth	10	Android importa PNG cifrado, descifra César y gestiona fallos
S1-06	Alta	Kenneth	6	Unity dispone de tutorial, enemigo, puerta y rondas base

Criterio de salida Sprint 1: las cuatro aplicaciones disponen de un camino funcional demostrable, existen PR revisadas y se ejecutan localmente las pruebas básicas pertinentes. El workflow de pruebas CI/CD se limita a Python. La integración aún podrá contener defectos que se resolverán en Sprint 2.

7.5. Backlog propuesto para Sprint 2 (semana 3; 48 h)

ID	Prior.	Resp.	h-pers.	Criterio de aceptación
S2-01	Alta	Koldo	8	Python: tests, cobertura ≥80 % en CI, CLI y empaquetado

ID	Prior.	Resp.	h-pers.	Criterio de aceptación
S2-02	Alta	Carlos	8	JavaFX: pruebas locales, cobertura ≥80 %, .txt/.png y distros Windows/Linux
S2-03	Alta	Kenneth	8	Android: accesibilidad, PNG/César, pruebas locales, cobertura ≥80 % y APK
S2-04	Alta	Kenneth	8	Unity: estados, 4 errores/vida, normalización, reinicio y test
S2-05	Alta	Carlos	8	Unity: pruebas locales, cobertura ≥80 %, físicas2D y ejecutable Windows 11 x64
S2-06	Alta	Koldo	8	Pruebas cruzadas R2AE, CI Python verde, 20 MiB, evidencias y manuales

Criterio de salida Sprint 2: los cuatro ejecutables/paquetes y la documentación pueden demostrarse; la cobertura unitaria ≥80 % de cada cliente se acredita con informes, pero solo Python tiene bloqueo automático por pruebas y cobertura en CI/CD. Los otros componentes aportan validaciones locales, pruebas funcionales e interoperabilidad. El cierre exige evidencias reales de ejecución e instalación, no una marca «Done» nominal.

7.6. Pair programming, Definition of Done y riesgos

Pair programming: sesiones de 2 h incorporadas dentro de las estimaciones de las tareas propuestas (no se suman por segunda vez). Durante Discovery, Carlos y Koldo revisarán el formato R2AE y Koldo con Kenneth ensayarán interoperabilidad; en Sprint 1, Carlos con Kenneth revisarán las rutas de JavaFX/Android y Kenneth con Koldo la normalización César; en Sprint 2, Carlos con Kenneth revisarán Unity y Carlos con Koldo los tests R2AE. Se rotarán driver y navigator a mitad de sesión y se anotarán ambas personas, conclusiones y commits en la Issue correspondiente tras crearla. Una PR de cambios sustanciales requiere revisión de alguien distinto de su autor.

Definition of Done: Issue con criterios verificables satisfechos; código integrado por PR revisada; tests unitarios y funcionales aplicables superados; workflow CI/CD de Python sin fallos; cobertura ≥80 % acreditada por cada aplicación, con informes locales donde corresponda; documentación actualizada, evidencia enlazada y ausencia de defectos bloqueantes conocidos.

Riesgo	Impacto	Medida preventiva / contingencia
R2AE incompatible entre clientes	Alto	Vectores cruzados desde Sprint 1; un único documento de bytes/cabecera
20 MiB / memoria insuficiente	Alto	Ensayo Discovery, perfilado y estrategia incremental/temporal si procede
CI Python o cobertura <80 %	Alto	Bloqueo en CI solo para Python; medición local del resto y corrección prioritaria
Pruebas locales Unity o compilación	Alto	Validar herramientas locales Edit/Play Mode y comprobar el ejecutable Windows
Disponibilidad inferior a 16 h/persona	Alto	Reestimar capacidad y dividir alcance; priorizar mínimos obligatorios
Ambigüedades PROM / requisitos A11Y	Medio	Consultar con docentes, anotar decisión acordada en Issue y actualizar contrato

8. Entregables y revisión contractual

8.1. Entrega al finalizar Discovery (4.9)

Se entregará en PDF un único contrato técnico corregido con responsables, arquitectura local, recorrido y UML, bocetos, estrategia prevista de tratamiento de ficheros, plan de pruebas y planificación Discovery + dos sprints. La copia .docx será un archivo editable de trabajo, no sustituye la entrega obligatoria en PDF. Antes del envío se confirmará la capacidad disponible. El tablero de GitHub Projects ya está disponible; las tablas documentales de las secciones 7.3 a 7.5 constituyen una posible organización que el equipo podrá ajustar al crear y priorizar sus Issues reales.

Discovery documentará el análisis previo de viabilidad, los riesgos y las limitaciones conocidas de memoria, almacenamiento y acceso, sin exigir resultados de ejecución antes de aprobar el contrato. Las pruebas reales sobre archivos de 0 B, 1 B, 1 MiB y al menos 20 MiB se realizarán durante el desarrollo y se acreditarán con la solución final. La aceptación exige cumplir el mínimo de 20 MiB; la técnica de implementación concreta no queda impuesta por el contrato.

8.2. Entrega final de aplicaciones

Componente	Entregables y alcance
Python	Código, dependencias, instrucciones y CLI con Vigenère/AES y gestión de archivos.
JavaFX	Código y fatjar Windows/Linux desde misma base; cifrado y descifrado AES.

Componente	Entregables y alcance
Android	Código Kotlin y APK; descifrado AES PNG y César .txt.
Unity 6.3	Proyecto, código y compilación ejecutable del minijuego 2D con estados y físicas.

8.3. Documentación y trazabilidad

- Repositorio GitHub con fuentes, dependencias, configuraciones, workflow CI/CD de Python, pruebas locales de los demás clientes y README.
- Manual de instalación y uso por aplicación; decisiones técnicas y justificación del formato R2AE.
- Limitaciones identificadas, incluyendo tamaño máximo real y compatibilidad.
- Tablero Scrum, incrementos de sprint, commits, ramas, Pull Requests y revisiones.

8.4. Evidencias técnicas y demostración

- Resultados reales de las pruebas AES en 0 B, 1 B, 1 MiB y ≥20 MiB, y recuperación con coincidencia de tamaño, SHA-256 y bytes.
- Incidencias: contraseña equivocada, archivo alterado, permisos, espacio y destino existente, con originales preservados.
- Pruebas de interoperabilidad R2AE entre Python, JavaFX y Android, con recuperación correcta de archivos compatibles.
- Informes de cobertura unitaria ≥80 % por aplicación, obtenidos localmente cuando corresponda, y ejecuciones de GitHub Actions únicamente para Python.
- Demostración de Python, JavaFX, Android y Unity, incluida validación de partida en curso, victoria y derrota.

8.5. Revisión contractual y aprobación

El Departamento de IT evaluará Discovery y comunicará «Aprobado» o «Requiere correcciones». Las observaciones deberán incorporarse antes de la aprobación; los cambios sustanciales posteriores en arquitectura, R2AE, tratamiento de ficheros y funcionalidades se justificarán.

La arquitectura propuesta ejecuta localmente su lógica en las cuatro aplicaciones y establece interoperabilidad mediante contenedores R2AE entre componentes compatibles. Se solicita la revisión conjunta con el profesorado, reconociendo que PROM forma parte del reto y solicitando aclaraciones de sus requisitos, así como de los datos de planificación que deban confirmarse y las decisiones pendientes sobre alcance, límites superiores y evaluación técnica. Cualquier cambio se resolverá por acuerdo documentado, sin pretender imponer condiciones unilaterales.

ESTADO: propuesta corregida para revisión conjunta. Existe un tablero GitHub Projects de Enigma; el backlog de Discovery y los dos sprints es orientativo y no representa una lista de Issues ya creadas. Siguen pendientes de confirmar la capacidad horaria y las aclaraciones del profesorado. La verificación real de ficheros de al menos 20 MiB corresponde al desarrollo posterior a la aprobación. La aprobación de Discovery no acredita todavía el producto final.

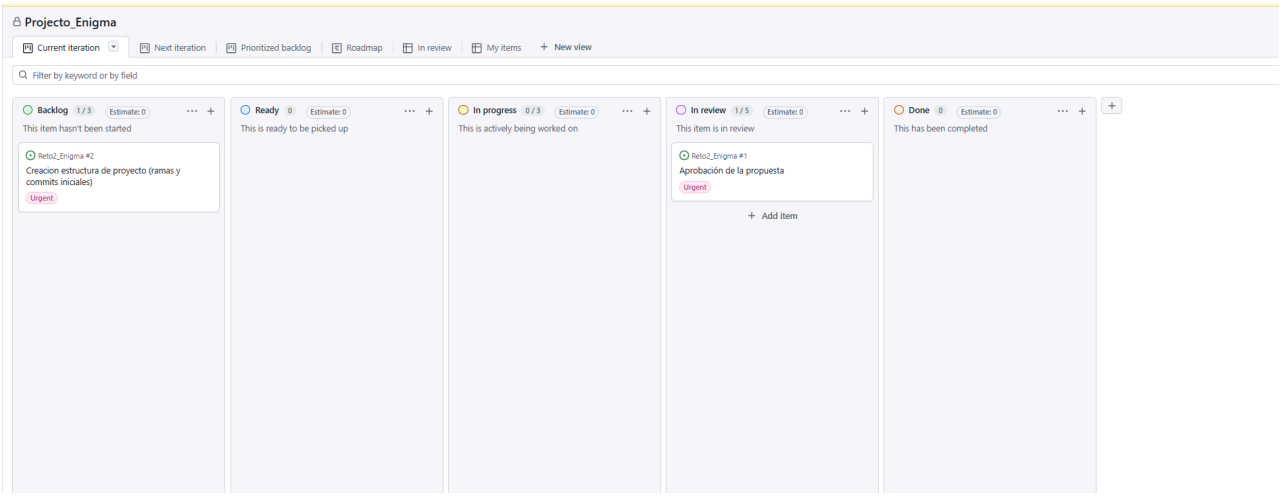
9. Requerimientos de los sistemas

Sistema	Version
Python	3.14
Java (JDK)	25 LTS (aplicación JavaFX)
Windows	11 de 64 bits (JavaFX y ejecutable Unity)

Sistema	Version
Linux	Ubuntu 26.04.1 LTS / Debian 13.1.0
Android	11
Android Studio	Quail 4 | 2026.1.4 (compilado el 28/08/2026)
Unity	6.3; compilación Windows x86-64

Anexo A. Estado actual del tablero de GitHub Projects

Captura realizada el 29/10/2026 de la vista Backlog. Se observan la Issue #1 «Aprobación de la propuesta» en «In progress» y la Issue #2 «Creación estructura de proyecto (ramas y commits iniciales)» en «Backlog». Las tablas de las secciones 7.3 a 7.5 continúan siendo una propuesta de organización, revisable por el equipo.



Extracto del estado actual del tablero de Enigma en GitHub Projects.